IOU Exchange

PARTNER API INTEGRATION HANDBOOK

Document IOUX-PARTNER-API-001 · v1.0
Audience API partner platforms & tech teams
Shareable No production secrets in this document
Date 15 September 2026
API https://uzimax.onrender.com/api/v1
Owner IOUX engineering

Document ID: IOUX-PARTNER-API-001

Version: 1.0

Date: 15 September 2026

Audience: Any platform, fund, broker, or tech team integrating with IOUX via API

Product: IOU Exchange — trade receivables registry & securitisation management (`ioux.africa`)

Live API (current): `https://uzimax.onrender.com/api/v1`

Health check (no auth): `GET /api/v1/health`

> This document is **shareable**. It contains **no production secrets**.
> API keys, HMAC secrets, and org UUIDs are issued separately via a secure handoff after onboarding approval.

Related: Teams integrating the AfyaX platform may also use the AfyaX field-level API examples doc once credentials are issued (secure handoff).

---

1. What IOUX is (and is not)

IOUX IS	IOUX IS NOT
A registry for invoices / IOUs (unique `IOU-KE-…` IDs)	A bank or licensed payments processor
A workflow for opt-in / verification, SPV assignment, escrow **recording**	A mover of real cash (wallets are simulated until a licensed rail is added)
An API layer for partner platforms to push parties, invoices, and repayments	A replacement for your trading / commerce / claims system
A place for SPV / admin / supplier / buyer portals to complete financing steps	NSE listing or legal trust-deed execution (process support only)

Separate databases. Your system and IOUX never share a DB. You store and reconcile using IDs we return (`organisationId` , `uzimaPartyId` , `invoiceId` , `iouRegistryId` , `iouxTransactionId`).

---

2. Who does what in the partnership

2.1 Roles in plain language

ROLE	WHO TYPICALLY	RESPONSIBILITY
Partner platform	e.g. AfyaX, Mint-linked systems, other B2B apps	Originate buyers/suppliers/trades; push clean data to IOUX; receive purchase confirmation; push repayments
IOUX	Uzima Exchange / IOU Exchange	Issue party & IOU IDs; run lifecycle; host portals; call your purchase webhook at disbursement
Supplier (originator)	Clinic, pharmacy, wholesaler, provider	Opts in (or lists) via IOUX portal / agreed path

ROLE	WHO TYPICALLY	RESPONSIBILITY
Buyer (obligor)	Insurer, hospital, corporate buyer	Owes the invoice; commitment to pay; may verify on supplier-listed path
SPV	IOU Exchange Capital SPV (or nominated vehicle)	Buys the receivable; records disbursement in IOUX escrow
Settlement agent / PSP	Your bank, M-Pesa, or licensed processor	Moves real money; IOUX only **records** outcomes

2.2 Responsibility matrix

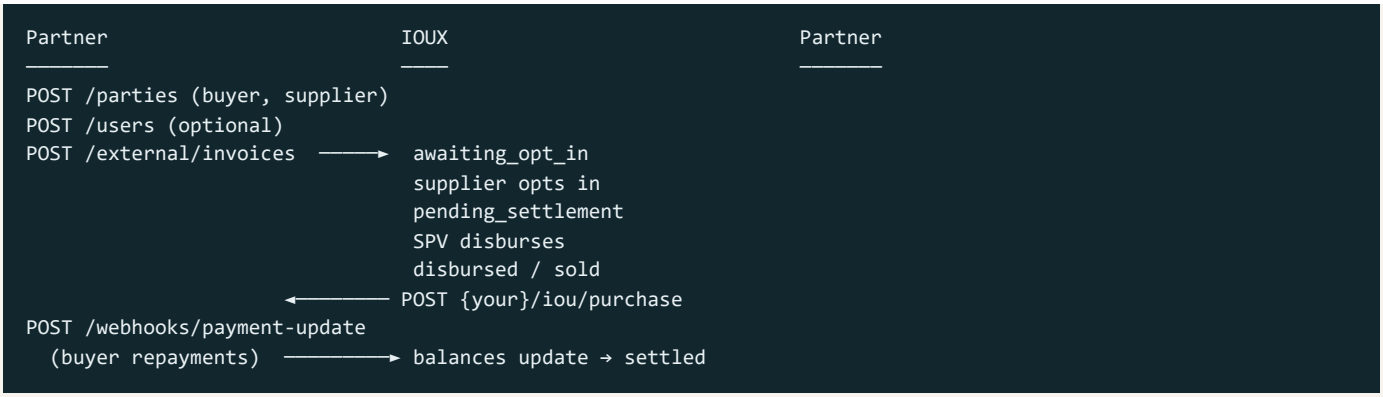
ACTIVITY	PARTNER PROVIDES	IOUX PROVIDES
Legal / commercial go-ahead to integrate	Signed intent / PoC approval from your side + UzimaX	Platform access policy + this handbook
Technical counterpart named	One engineer + backup contact	One IOUX engineer for UAT
Environments	Your sandbox + production base URLs	IOUX UAT/prod API base + portal URL
Credentials	Secure vault on your side	Platform API key, scopes, HMAC secret, platform org UUID (secure channel)
Buyer & supplier master data	Legal name, reg/KYC fields you hold, your external IDs	`uzimaPartyId` + org UUID
Users who must use IOUX portal	Emails + roles (supplier checker, etc.)	Invite emails / temp passwords (optional)
Invoice / IOU commercial data	Face value, dates, parties, your invoice/order IDs, docs URLs	`iouRegistryId`, status machine, registry search
Asking / list price semantics	Confirm what "asking price" means on your listing	Send purchase webhook `amount` = listed/asking amount (not net discount only)
Purchase notification endpoint	HTTPS URL that accepts IOUX purchase payload	Outbound `POST` on **disbursement only** (DvS)
IP allowlisting / auth on your webhook	Whitelist IOUX egress and/or shared secret policy you choose	Stable event payload + retries logged
Supplier opt-in / buyer verify	Users ready and briefed	Portal screens + notifications
SPV disbursement	—	Escrow release in IOUX (recorded)
Buyer repayments	Signed payment-update webhooks	Balance / status updates on the instrument
UAT evidence	Sample payloads + your logs	Webhook delivery logs + registry IDs
Go-live sign-off	Written UAT acceptance	Checklist + production enablement

3. Integration models we support

Pick **one primary model** before build. Do not mix casually.

Model A — Partner originates credit sales (most common)

Partner's marketplace / credit flow creates the receivable. Partner pushes parties + invoice to IOUX. Supplier **opts in** on IOUX. SPV disburses on IOUX. IOUX notifies partner that the receivable was purchased.



Model B — Supplier lists on IOUX; buyer verifies

Same APIs with `submissionType: "supplier_list"`. Status starts at `awaiting_buyer_verification`. Buyer confirms in portal, then assignment / settlement path continues.

Model C — Portal-only (no partner API)

For early PoCs with a few providers: IOUX issues portal logins only. No API. Useful for Mint-style trials before full system integration. Escalate to Model A when volumes justify it.

---

4. What we expect from you (partner checklist)

Before we issue production credentials, please provide:

4.1 Organisation & contacts

- [ ] Legal entity name and primary commercial contact
- [ ] Technical lead name, email, phone
- [ ] Escalation contact for UAT week
- [ ] Intended go-live window and approximate monthly IOU volume

4.2 Environments

- [ ] Sandbox base URL (if you expose one)
- [ ] Production base URL
- [ ] Exact **purchase webhook path** (e.g. `/api/v1/iou/purchase`)
- [ ] Confirmation whether auth is IP allowlist, shared secret, mTLS, or open+HTTPS

4.3 Data readiness

- [ ] How you identify buyers and suppliers internally (external IDs)
- [ ] Sample JSON for: one buyer, one supplier, one invoice/IOU
- [ ] Confirmation of currency (default **KES**) and max tenor you will send (IOUX programmes may cap, e.g. **120 days**)
- [ ] What field is your **asking / list price** (must match purchase webhook amount validation on your side)
- [ ] Whether documents are public HTTPS URLs or need alternate intake

4.4 People for first UAT

- [ ] At least one **supplier** user who can log into IOUX and opt in
- [ ] At least one **buyer** party already known in your system
- [ ] Agreement that SPV disbursement will be done by IOUX operators during UAT

4.5 Security

- [ ] Secret storage location (vault / env) confirmed — no secrets in chat
- [ ] Ability to sign outbound payment webhooks (HMAC)

- [ ] Ability to receive and log IOUX purchase callbacks

5. What IOUX will provide to you

After approval, you receive (secure channel — not in this PDF/handbook copy):

ITEM	PURPOSE
API base URL	All Partner → IOUX REST calls
Platform API key (`uzima_…`)	Auth header `X-API-Key` or `Authorization: Bearer`
Platform org UUID / party ID	Your platform identity on IOUX
HMAC secret	Sign `POST /webhooks/payment-update`
Portal URL	Human workflows (opt-in, escrow, registry)
Demo / UAT operator accounts (if agreed)	SPV / admin assisted testing
Webhook delivery visibility	IOUX logs HTTP status from your purchase endpoint
This handbook + OpenAPI / Postman (where published)	Implementation reference
Named IOUX engineer for joint UAT window	Debug + payload review

IOUX also provides operationally during UAT:

- Creation / enablement of your platform integration row (purchase URL, environment)
- Confirmation when outbound purchase webhook is configured
- Joint walkthrough of one happy-path IOU end-to-end
- Written note of open items (business rules vs transport)

6. Authentication

6.1 Partner → IOUX (API key)

```
X-API-Key: uzima_<issued-to-you>
Content-Type: application/json
```

or

```
Authorization: Bearer uzima_<issued-to-you>
```

Scopes typically include: `parties:write` , `parties:read` , `invoices:write` , `payments:write` (exact set on your key).

6.2 Partner → IOUX payment updates (HMAC)

`POST /api/v1/webhooks/payment-update` requires:

HEADER	VALUE
`X-Afyax-Timestamp`	Unix **seconds** (or ms — both accepted)
`X-Afyax-Signature`	`hex(HMAC_SHA256(secret, "{timestamp}.{rawBody}"))` or `sha256=<hex>`
`X-API-Key`	Your platform key

Sign the **exact raw JSON body** bytes. Clock skew beyond the allowed window → `401` .

6.3 IOUX → Partner (purchase webhook)

Default model: HTTPS POST JSON to your purchase URL.

You should:

1. Accept JSON body (see §8)
2. Validate listing still **available** and `amount` ≥ / = your asking price
3. Return `2xx` on success; `4xx` with clear JSON error on business rejection
4. Be idempotent on repeated delivery of the same `ioux_id` / transaction ref

---

7. Core API map (Partner → IOUX)

#	METHOD & PATH	YOU USE IT TO…
1	`POST /parties`	Register / upsert buyer or supplier
2	`GET /parties/{uzimaPartyId}`	Read back a party
3	`POST /users`	Create IOUX portal user under an org
4	`POST /external/invoices`	Submit credit sale / receivable (creates IOU)
5	`POST /ious`	Alias of invoice create
6	`GET /ious/{iouRegistryId}`	Poll status
7	`GET /invoices/{id}/status`	Poll by invoice UUID
8	`POST /webhooks/payment-update`	Report buyer repayment

Full field schemas: see OpenAPI (`docs/openapi.yaml`) and AfyaX worked examples (`docs/AFYAX_API_FOR_SULE.md`).

7.1 Minimum invoice fields we need

FIELD	REQUIRED	NOTES
Buyer identity	Yes	`buyerOrgId` **or** `buyerPartyId`
Supplier identity	Yes	`supplierOrgId` **or** `supplierPartyId`
`faceValue`	Yes	Positive number
`issueDate` , `dueDate`	Yes	`YYYY-MM-DD` ; tenor limits may apply
`commitmentToPay`	Yes for credit sale	Should be `true` when buyer owes
`listedAmount`	Strongly recommended	Your **asking / list price**; drives purchase webhook amount
`externalInvoiceId`	Recommended	Your ID for reconciliation
`submissionType`	Optional	`buyer_credit_sale` (default) or `supplier_list`
Supporting docs	Recommended	`{ name, fileUrl, docType }`

Response you must store: `invoiceId` , `iouRegistryId` , initial `status` .

---

8. IOUX → Partner purchase webhook (critical)

8.1 When it fires

Only on `iou.disbursed` — after SPV records disbursement to the supplier (Delivery vs Settlement).

It does **not** fire at invoice create or at supplier opt-in alone.

8.2 Example payload

```
{
  "buyer_name": "IOU Exchange Capital SPV",
  "ioux_id": "IOU-KE-2026-00006-7",
  "payment_method": "bank",
  "amount": 5830.00,
  "transaction_id": "IOUX-TXN-8c449a779bd5",
  "bank_reference": "IOUX-TXN-8c449a779bd5"
}
```

FIELD	MEANING
`buyer_name`	**SPV / purchaser of the receivable on IOUX** (not the pharmacy/obligor)
`ioux_id`	Same as `iouRegistryId`
`amount`	Asking / listed amount (align with your listing price validation)
`transaction_id` / `bank_reference`	IOUX settlement reference for reconciliation

8.3 What we expect your endpoint to do

1. Find listing by `ioux_id`
2. Ensure listing is still **open / available**
3. Validate `amount` against asking price
4. Mark sold / transferred; notify your users if needed
5. Return success JSON

Common UAT failures (learn from AfyaX pilot):

YOUR HTTP RESPONSE	TYPICAL CAUSE	FIX
400 amount < asking price	IOUX sent discounted net; you validate face/ask	Align on listed/asking amount
400 listing already sold	Listing closed before webhook, or retry after success	Keep listing open until purchase accepted; make handler idempotent
Timeout / 5xx	Endpoint down or IP blocked	Whitelist + monitoring

IOUX retries failed deliveries; design for **at-least-once** delivery.

9. Lifecycle statuses (what you will see)

STATUS	MEANING
`awaiting_opt_in`	Buyer-originated credit sale waiting for supplier
`awaiting_buyer_verification`	Supplier-listed path waiting for buyer
`pending_settlement`	Accepted; assignment recorded; payment not yet disbursed
`disbursed`	SPV disbursement recorded; purchase webhook sent
`settled`	Repayments brought outstanding to closure (no purchase webhook)
`opt_in_declined` / `buyer_rejected` / `cancelled`	Closed without purchase

Listing flag `sold` on IOUX means the receivable was acquired on our side.

10. Joint UAT script (use with any new partner)

Goal: One clean IOU from create → purchase accept on your side.

STEP	ACTOR	ACTION	PASS CRITERIA
0	Both	Confirm API key + purchase URL configured	`GET /health` ok; test auth 401 without key
1	Partner	`POST /parties` buyer + supplier	`uzimaPartyId` returned; store both
2	Partner	`POST /users` for supplier checker (optional)	Invite received / login works
3	Partner	`POST /external/invoices` with `listedAmount` = asking price	`iouRegistryId` + `awaiting_opt_in`
4	Supplier	Opt in on IOUX portal	Status → `pending_settlement`
5	SPV / IOUX ops	Release disbursement escrow leg	Status → `disbursed`
6	Partner	Receive purchase webhook	`2xx`; listing marked sold **once**
7	Partner	`POST /webhooks/payment-update` sample repayment	`iouxTransactionId` returned
8	Both	Sign UAT note	Open items listed explicitly

Do not reuse a listing that you already marked sold for a fresh purchase test. Always create a **new** IOU / listing.

---

11. Onboarding process (how we start with you)

1. **Commercial intro** — UzimaX / Githuku introduces the partner.
2. **Scope call** — Choose Model A/B/C; name technical leads.
3. **You return §4 checklist** (contacts, URLs, sample JSON).
4. **IOUX provisions** platform org + API key + HMAC (secure handoff).
5. **Configure** your purchase webhook URL on IOUX integrations.
6. **Sandbox / prod UAT** using §10 script (target: one clean pass).
7. **Written sign-off** + production monitoring.
8. **Operate** — support channel for incidents; rotate secrets on schedule.

Typical first joint UAT window: **half-day to two days** if data and users are ready.

---

12. Security & compliance expectations

- Never commit API keys or HMAC secrets to git or group chats.
- Use HTTPS only.
- Prefer least-privilege scopes on API keys.
- Log webhook receipts with `ioux_id` and your listing id for audit.
- IOUX does not store card PANs; keep payment card data in your PCI scope.
- KYC documents: send metadata + URLs you control; agree retention with UzimaX legal if required.
- Personal data: process under your privacy notice and applicable Kenya Data Protection Act obligations.

---

13. What IOUX will not do for you

- Rebuild your commerce / claims / underwriting product
- Guarantee your listing remains unsold if your own system closes it early
- Move real money without a licensed processor integration (separate scope)
- Provide formal legal opinions, trust deeds, or NSE listing fees
- Unlimited unpaid parallel “dummy” environments without an agreed UAT plan

14. Support & escalation

TOPIC	CONTACT PATTERN
Commercial / PoC scheduling	UzimaX commercial lead (e.g. Githuku / Jugo Limited)
API / webhook technical	Named IOUX engineer (issued at onboarding)
Partner platform bugs	Your technical lead
Production outage	Email + phone escalation list exchanged at go-live

When reporting an issue, include: `iouRegistryId`, UTC timestamp, request/response bodies (redact secrets), and whether the failure is **transport** (5xx/timeout) or **business rule** (4xx with message).

15. Glossary

TERM	MEANING
IOU / receivable	Right to receive payment recorded in the registry
`iouRegistryId`	Canonical public ID, e.g. `IOU-KE-2026-00006-7`
Asking / listed amount	Price your platform treats as the purchase amount
DvS	Delivery versus settlement — purchase notify at disbursement
SPV	Special purpose vehicle that acquires the receivable
Obligor	Buyer who owes on the invoice
Originator	Supplier who is owed / may sell the receivable

16. Document control

VERSION	DATE	NOTES
1.0	15 Sep 2026	First shareable partner handbook (post AfyaX pilot lessons)

Owner: IOUX engineering / Alfred Mulinge

Approved for sharing with: Prospective API partners under NDA or PoC letter as required by UzimaX

Appendix A — Quick “bring this to the kickoff” pack

Send IOUX these five items to start fast:

1. Tech lead contact
2. Purchase webhook URL (sandbox)
3. One sample buyer JSON + one sample supplier JSON
4. One sample invoice JSON with asking price field identified
5. Two named humans for first UAT (supplier user + your engineer)

We will return:

1. API base + key + HMAC (secure)
2. Portal links + any UAT logins
3. Scheduled UAT slot
4. This handbook acknowledged as the working contract for technical expectations

Appendix B — Email template you can forward

Subject: IOUX partner API onboarding — what we need from you

> Please find the IOUX Partner API Integration Handbook.
>
> In short: you push parties and invoices into IOUX; your users complete opt-in/verify on the IOUX portal where required; when the SPV disburses, we call your purchase webhook; you later push repayments.
>
> Reply with Appendix A items so we can issue credentials and book a joint UAT.
>
> Handbook: *(attach or link IOUX-PARTNER-API-001)*